

Join the lecture online on your dashboard.

NNs in Computer Vision: Convolutional Neural Networks

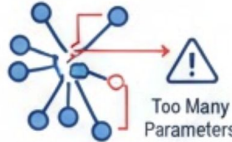
Agenda

We'll cover:

1. Limitations of ANN for Vision
2. Convolution operation
3. Parameters: Kernel size, Stride, Dilation
4. Dimensions and Pooling
5. CNN Architecture

01. THE PROBLEM

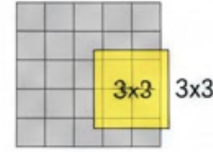
Limitations of ANN



Too Many Parameters

02. THE CORE

Convolution Operation



3x3 3x3

03. THE KNOBS

Parameters

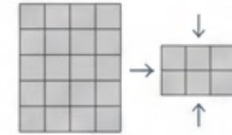


Kernel Stride Dilation

$$\text{Output Size} = \left\lceil W - \frac{W - K + 2P}{K + 2P} + 1 \right\rceil$$

04. THE REDUCTION

Dimensions & Pooling


$$\text{Output Size} = \left\lceil W - \frac{W - K + 2P}{S + S} + 1 \right\rceil$$

05. THE RESULT

CNN Architecture



Input Image

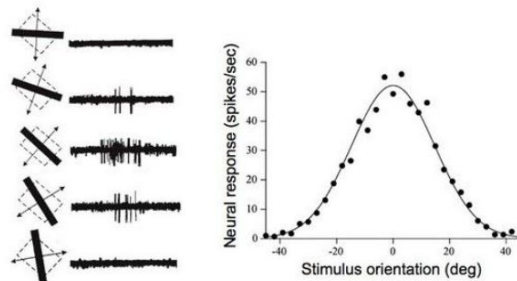
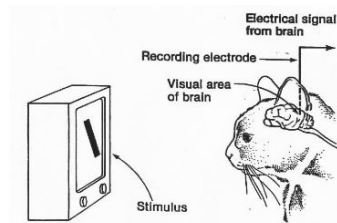
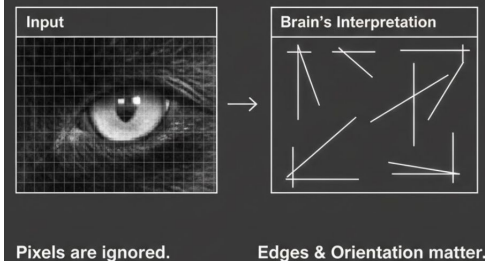
SBW P10K F08L F08L P8K P8K FC - S6FTMAX

Background: Neuroscience

History and motivation

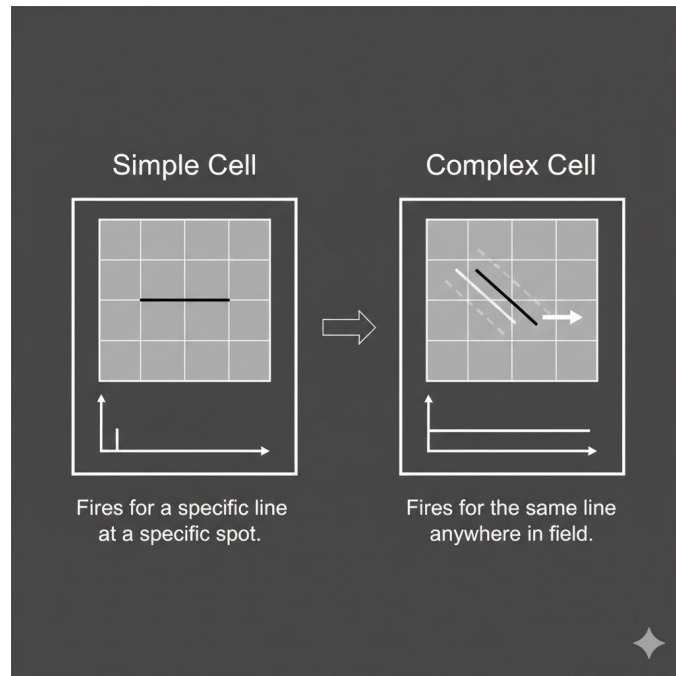
- Hubel and Wiesel found out that the brain doesn't care about pixels.
- It cares about edges and orientation.

Hubel and Wiesel Experiment



History and motivation

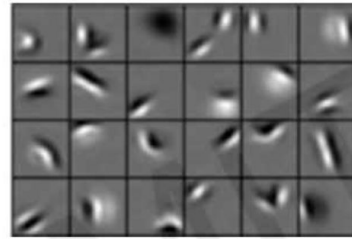
- They found two types of cells:
 - Simple cells fired only when they saw a line at a specific angle in a specific spot.
 - Complex cells fired if they saw the same line anywhere in the receptive field.



Findings of the experiment

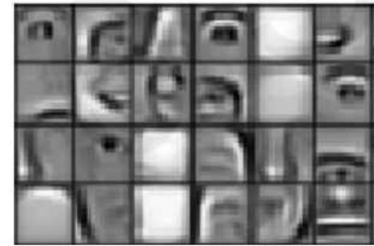
- Visual cortex is a layered system.
- Simple cells identify lines and edges at a specific position.
- Complex cells sees then anywhere in the receptive field (translation invariance).
- More complex cells combine these to form simple shapes.
- In the later layers these shapes combine to form complex objects.

Low level features



Edges, dark spots

Mid level features



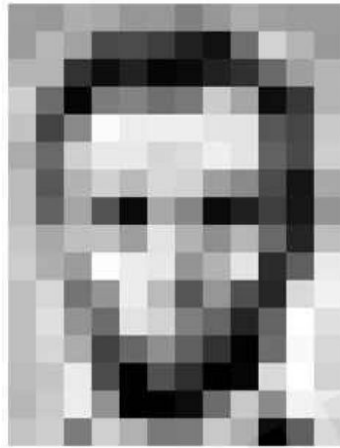
Eyes, ears, nose

High level features



Facial structure

Tasks in Computer Vision



Input Image



157	153	174	168	150	152	129	161	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	5	10	33	48	106	169	181
206	109	5	124	131	111	120	204	166	15	56	180
154	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	168
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	199	198	227	178	143	182	106	36	190
205	174	195	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	179	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	128	207	177	121	123	200	175	13	96	218

Pixel Representation

classification

Lincoln

0.8

Washington

0.1

Jefferson

0.05

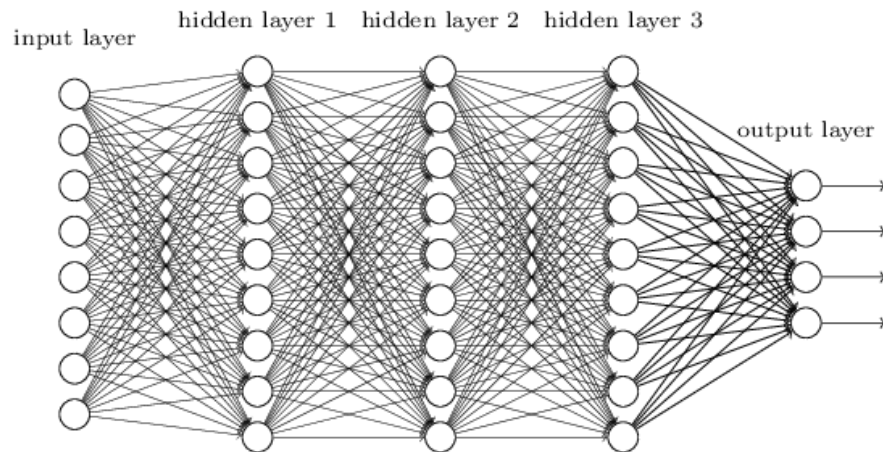
Obama

0.05

- **Regression:** output variable takes continuous value
- **Classification:** output variable takes class label. Can produce probability of belonging to a particular class

Using ANN for Image Processing

- Why can't we just flatten the image and feed to an ANN?



The Computational Bottleneck of Fully Connected Layers

Standard neural networks define a weight between every input pixel and every hidden unit. For high-dimensional data like images, this creates an exponential explosion in complexity.

Input Dimensions: 224×224 RGB image = **150,528 inputs**

Result: Massive overfitting & memory failure.



- ⚠️ ****Structural Flaw*:** No concept of spatial locality. Pixel (0,0) is treated the same as Pixel (0,200).
- ⚠️ ****Stability Flaw*:** A shift of one pixel changes every input value, forcing relearning.

Limitations...

- **Spatial relation:**

A fully connected network doesn't have any concept of neighborhood & treats all inputs equally.

Nearby image pixels are statistically related (e.g. the placement of eye pixels above nose pixels).

- **Translational and Rotational Invariance:**

Consider the image of a tree and we shift the tree in the image, all the pixel values change, but it is still a tree.

ANNs can't handle this translation (transformation).

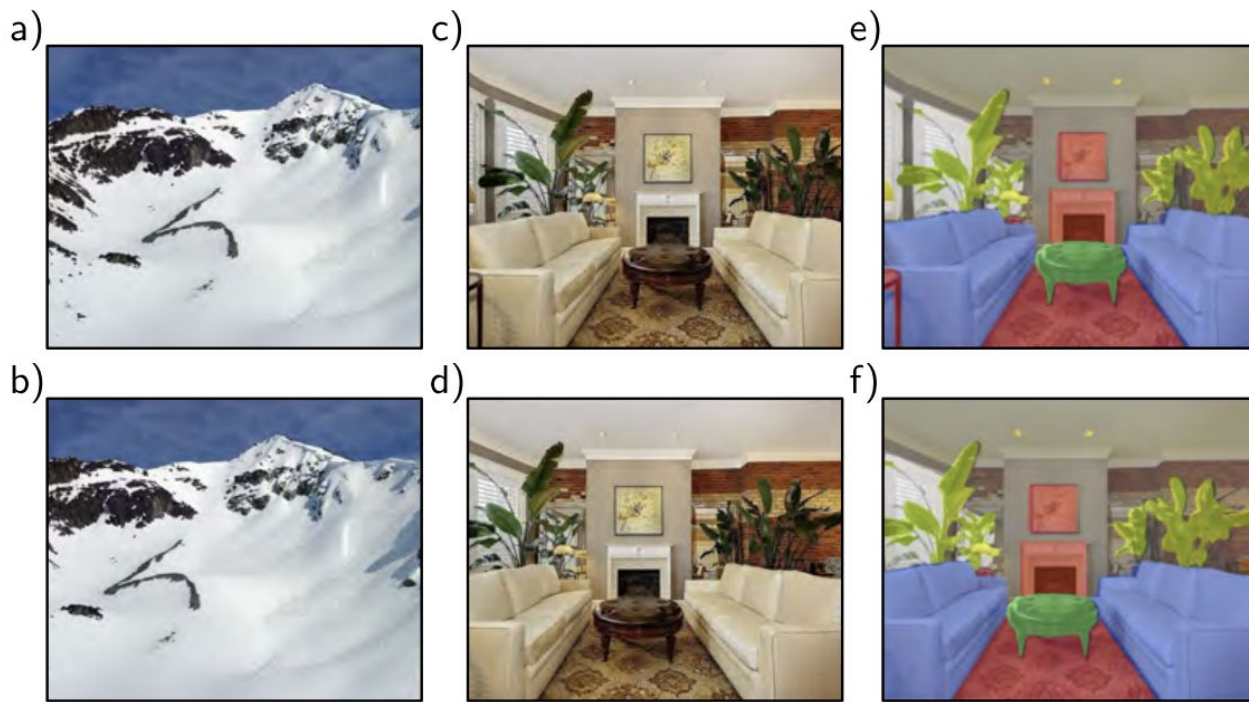


Figure 10.1 Invariance and equivariance for translation.

a–b) In image classification, the goal is to categorize both images as “mountain” regardless of the horizontal shift that has occurred. In other words, we require the network prediction to be invariant to translation.

c,e) The goal of semantic segmentation is to associate a label with each pixel.

d,f) When the input image is translated, we want the output (colored overlay) to translate in the same way. In other words, we require the output to be equivariant with respect to translation.

Panels c–f) adapted from Bousseth et al. (2021).

Conclusion

- It's computationally inefficient to train so much neurons.
- Solution:
 - We need to somehow detect features without looking at all the pixels simultaneously.
 - Exploit image spatial structure, learn patches and build them up.

Feature Extraction

How did you identify this as the bird?



Did you focus
on the
background?

High level feature detection

- Let's identify key features in each image category



Nose, Eyes, Mouth



Wheels, License Plate,
Headlights



Door, Windows, Steps

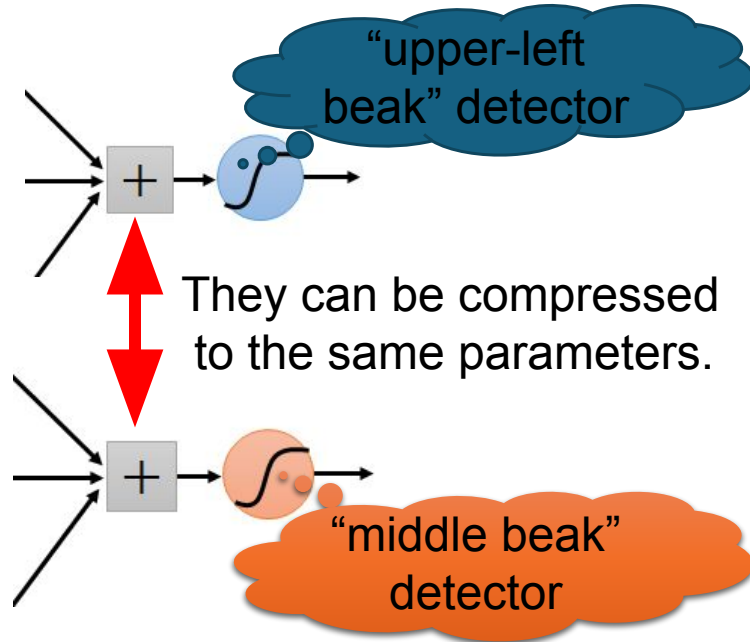
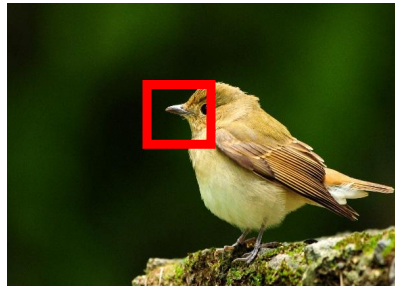
Consider learning an image

- We identify an image as a set of smaller features and not as a set of pixels.
- Some patterns are much smaller than the whole image

Can we represent a small region with fewer parameters?



Same pattern appears in different places:
What about training a lot of such “small” detectors
and each detector must “move around”.



Manual Feature Extraction

Domain knowledge

Define features

Detect features
to classify

Viewpoint variation



Scale variation



Deformation



Occlusion



Illumination conditions



Background clutter



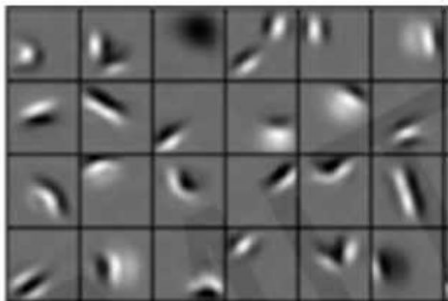
Intra-class variation



Learning Feature Representations

Can we learn a **hierarchy of features** directly from the data instead of hand engineering?

Low level features



Edges, dark spots

Mid level features



Eyes, ears, nose

High level features



Facial structure

Design Objectives: Invariance vs. Equivariance

Invariance (Classification):

Output remains constant under transformation.

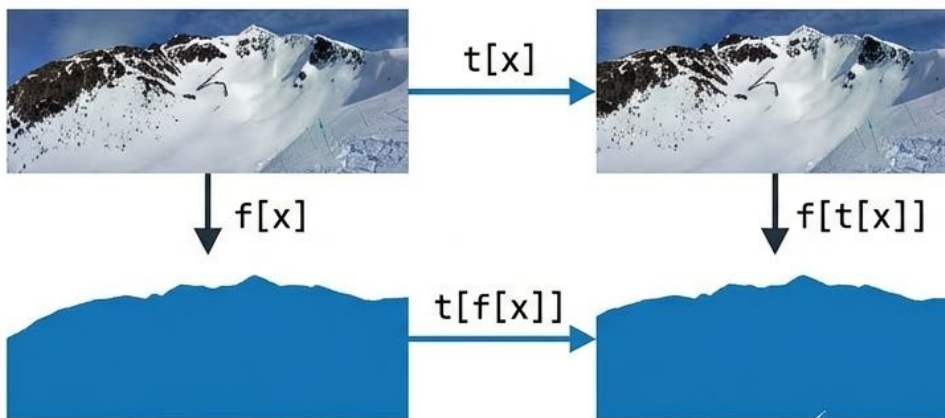
$$f[t[x]] = f[x]$$



Equivariance (Segmentation):

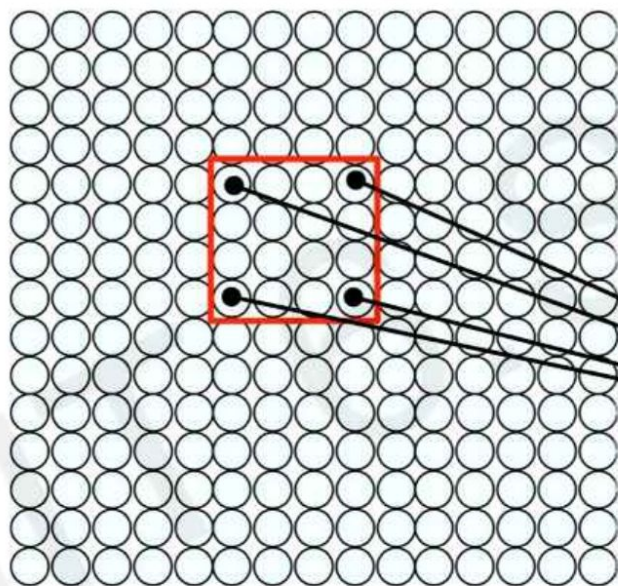
Output transforms in the same way as the input.

$$f[t[x]] = t[f[x]]$$



Using Spatial Structure

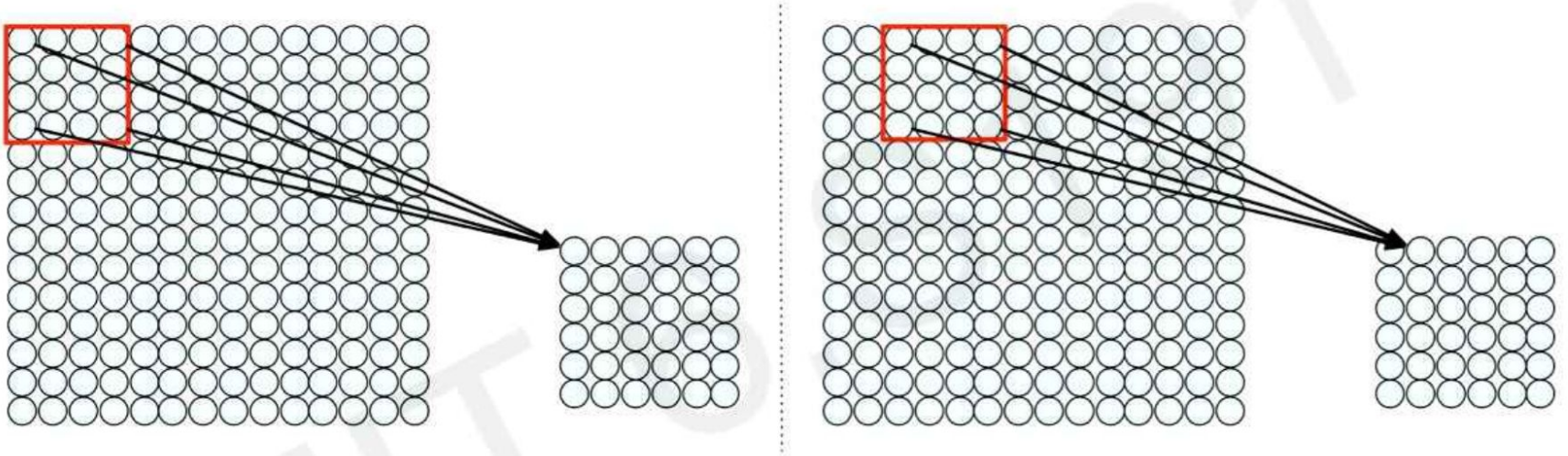
Input: 2D image.
Array of pixel values



Idea: connect patches of input
to neurons in hidden layer.
Neuron connected to region of
input. Only "sees" these values.

Convolution Operation

Using Spatial Structure



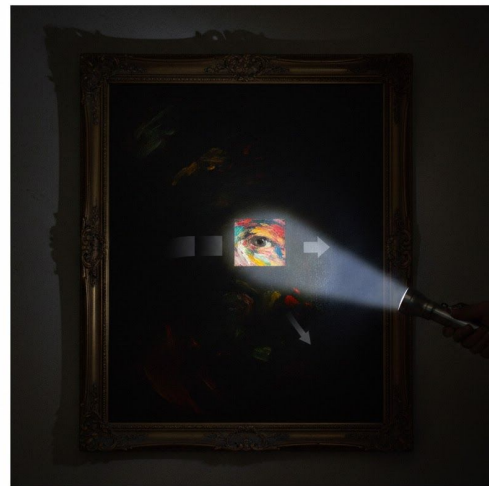
Connect patch in input layer to a single neuron in subsequent layer.

Use a sliding window to define connections.

How can we **weight** the patch to detect particular features?

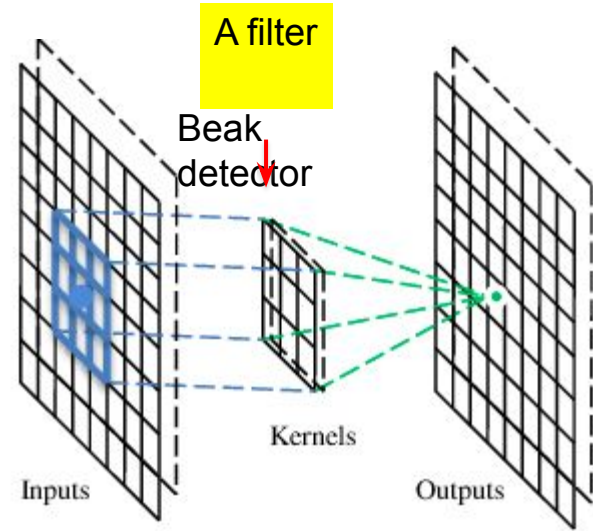
Convolution

- Imagine a large painting in a dark room.
- We need to see what is in the painting.
- But we have a small flashlight, which can illuminate only a certain portion of an image.
- We can slide the illuminated area from right to left and top to bottom to gain insight.



A convolutional layer

- A CNN is a neural network with some convolutional layers (and some other layers).
- A convolutional layer has a number of filters that does convolutional operation.



Types of convolution

- Based on the dimensionality, we have following types of convolution:
 - 1D convolution
 - 2D convolution
 - 3D convolution

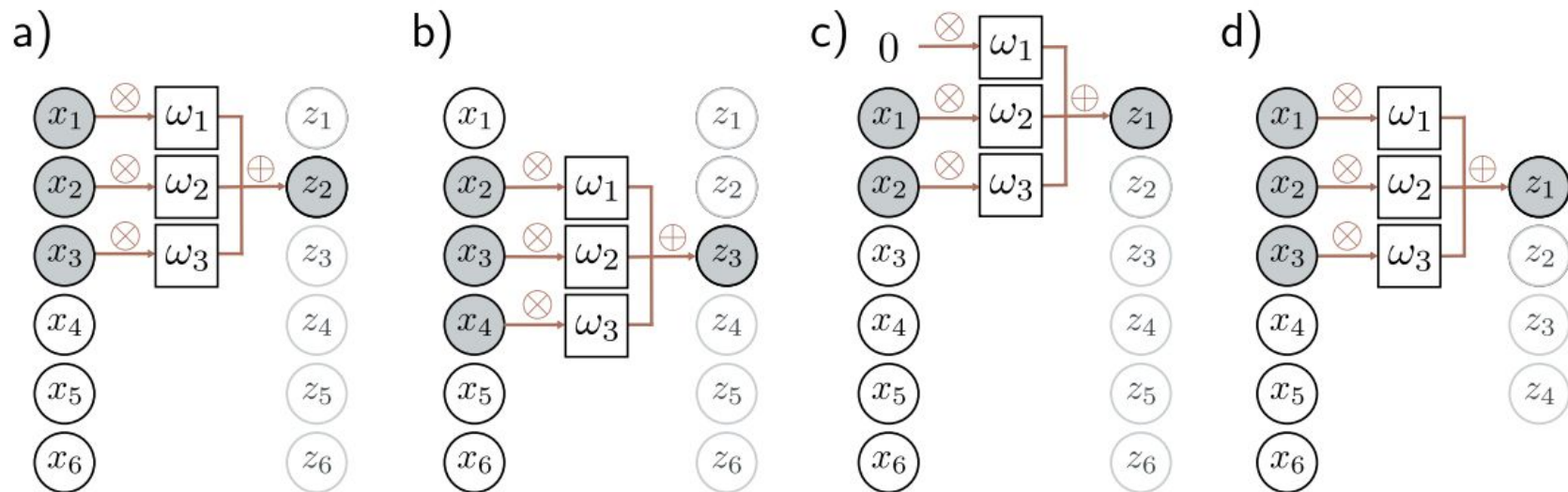
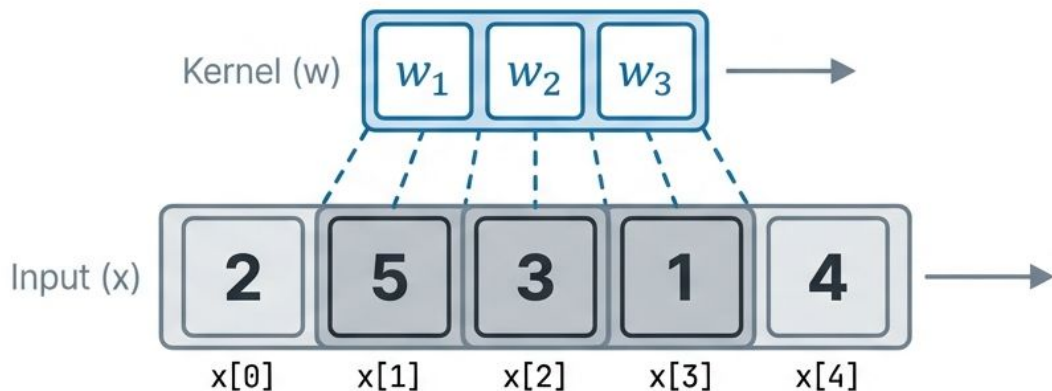


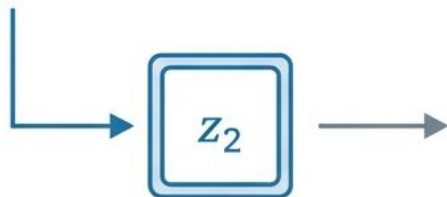
Figure 10.2 1D convolution with kernel size three. Each output z_i is a weighted sum of the nearest three inputs x_{i-1} , x_i , and x_{i+1} , where the weights are $\omega = [\omega_1, \omega_2, \omega_3]$. a) Output z_2 is computed as $z_2 = \omega_1 x_1 + \omega_2 x_2 + \omega_3 x_3$. b) Output z_3 is computed as $z_3 = \omega_1 x_2 + \omega_2 x_3 + \omega_3 x_4$. c) At position z_1 , the kernel extends beyond the first input x_1 . This can be handled by zero-padding, in which we assume values outside the input are zero. The final output is treated similarly. d) Alternatively, we could only compute outputs where the kernel fits within the input range ("valid" convolution); now, the output will be smaller than the input.

The 1D Convolution Operation



Output z_i calculation at position $i=2$:

$$z_2 = (w_1 \cdot x[1]) + (w_2 \cdot x[2]) + (w_3 \cdot x[3]) = (w_1 \cdot 5) + (w_2 \cdot 3) + (w_3 \cdot 1)$$



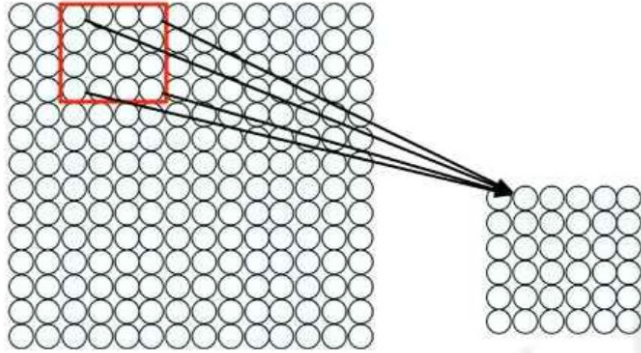
Engineering Concept: Parameter Sharing

Unlike FC layers, the same set of weights (kernel) is used at every position.

→ Benefits:

- Drastic reduction in parameters;
- Translation equivariance.

Feature Extraction with Convolution

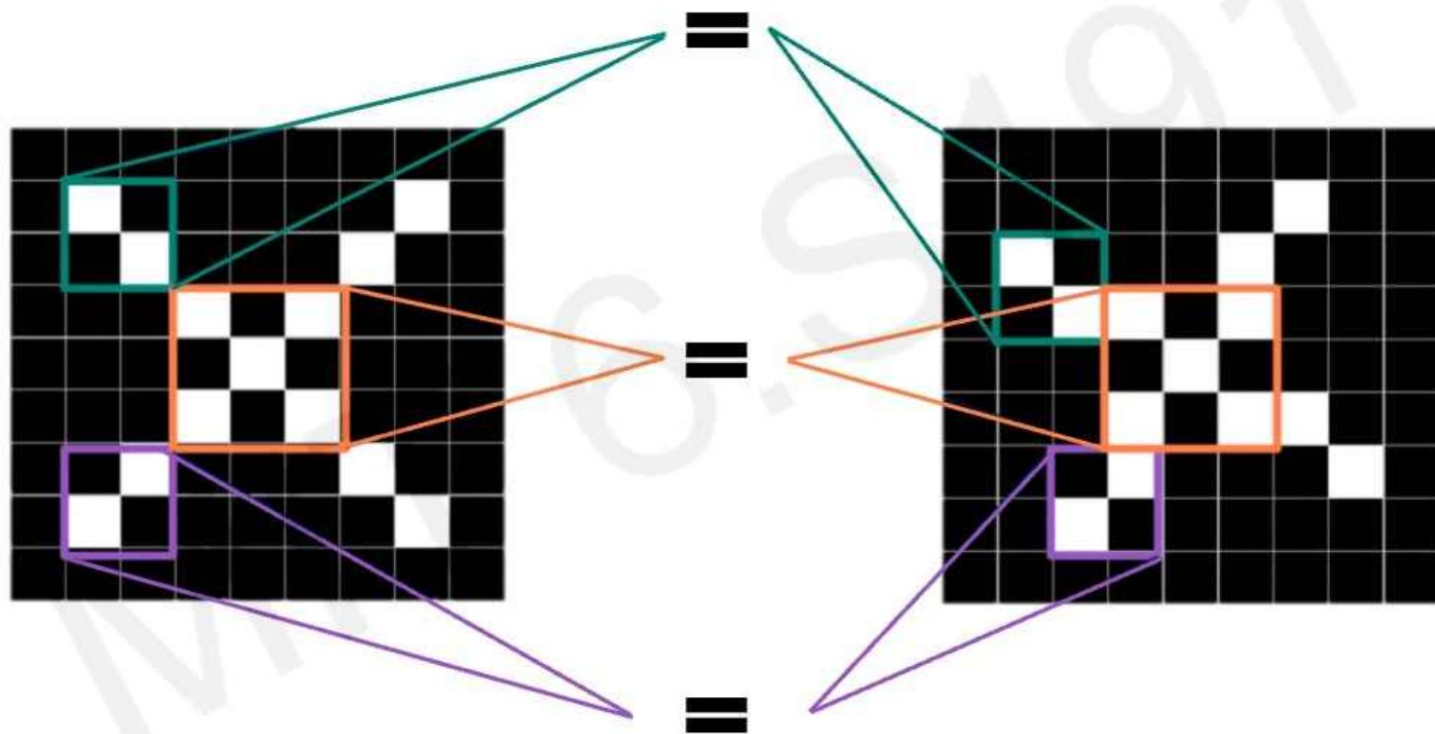


- Filter of size 4x4 : 16 different weights
- Apply this same filter to 4x4 patches in input
- Shift by 2 pixels for next patch

This “patchy” operation is **convolution**

- 1) Apply a set of weights – a filter – to extract **local features**
- 2) Use **multiple filters** to extract different features
- 3) **Spatially share** parameters of each filter

Features of X



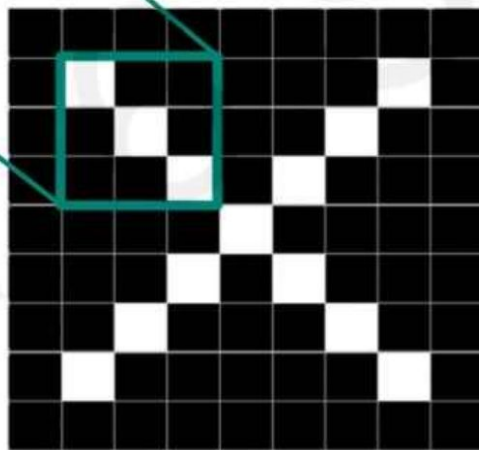
Filters to Detect X Features

filters

1	-1	-1
-1	1	-1
-1	-1	1

1	-1	1
-1	1	-1
1	-1	1

-1	-1	1
-1	1	-1
1	-1	-1



The Convolution Operation

$$\begin{bmatrix} 1 & -1 & -1 \\ -1 & 1 & -1 \\ -1 & -1 & 1 \end{bmatrix} \times \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = 1$$

element wise
multiply

add outputs

-1	-1	-1	1	-1	-1	-1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	-1	-1	1	-1	-1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	-1	-1	-1	-1	-1	-1	-1



1	1	1
1	1	1
1	1	1

= 9

The Knobs

Kernel size, Stride, Dilation

- **Kernel** size is the size of input region for which weighted sum is calculated.
- **Stride** is the amount of shift between subsequent convolution.
- **Dilation** rate is determined by the number of 0s interspersed between kernel weights.
- Dilation rate of 1 is the normal convolution.
- Dilation rate of 2 skips 1 pixel (i.e. insert one 0 between kernel weights).

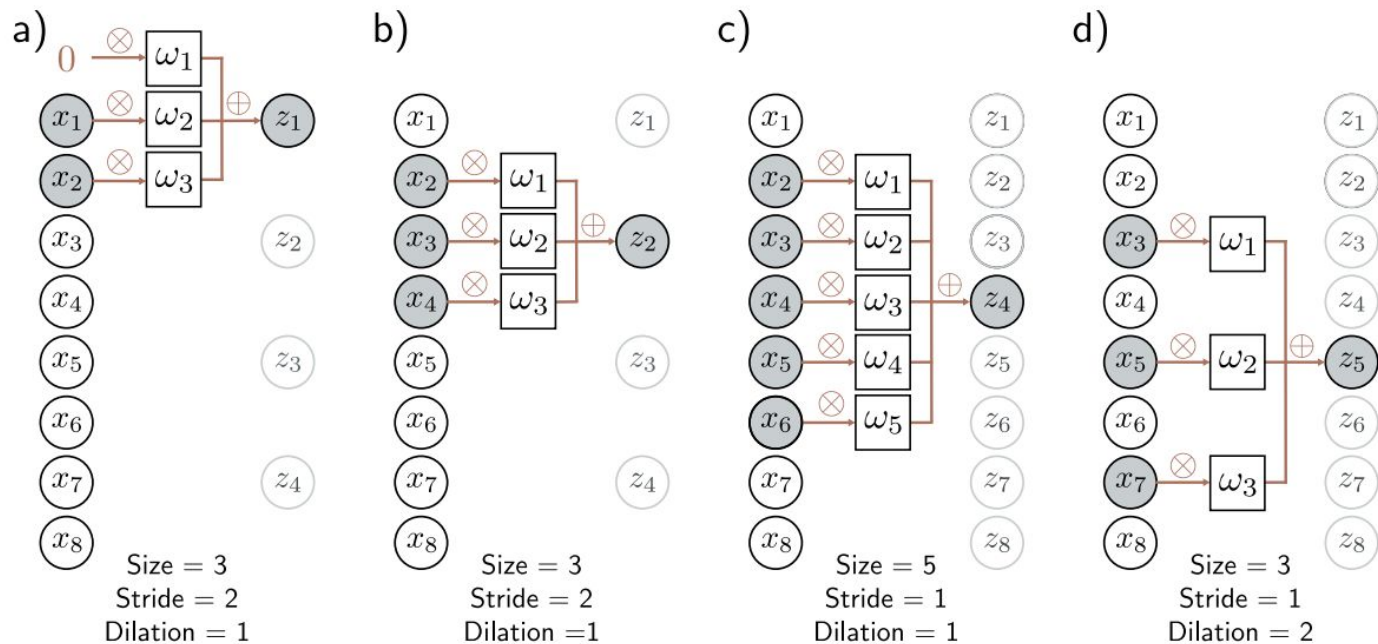


Figure 10.3 Stride, kernel size, and dilation. a) With a stride of two, we evaluate the kernel at every other position, so the first output z_1 is computed from a weighted sum centered at x_1 , and b) the second output z_2 is computed from a weighted sum centered at x_3 and so on. c) The kernel size can also be changed. With a kernel size of five, we take a weighted sum of the nearest five inputs. d) In dilated or atrous convolution (from the French “à trous” – with holes), we intersperse zeros in the weight vector to allow us to combine information over a large area using fewer weights.

2D Convolution

These are the network parameters to be learned.

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

-1	1	-1
-1	1	-1
-1	1	-1

Filter 2

⋮ ⋮

Each filter detects a small pattern (3 x 3).

Filter

- The filter is basically a feature detector.
- It “lights up” (activates) when it sees a specific pattern (like a vertical edge or a curve)



Original



Sharpen



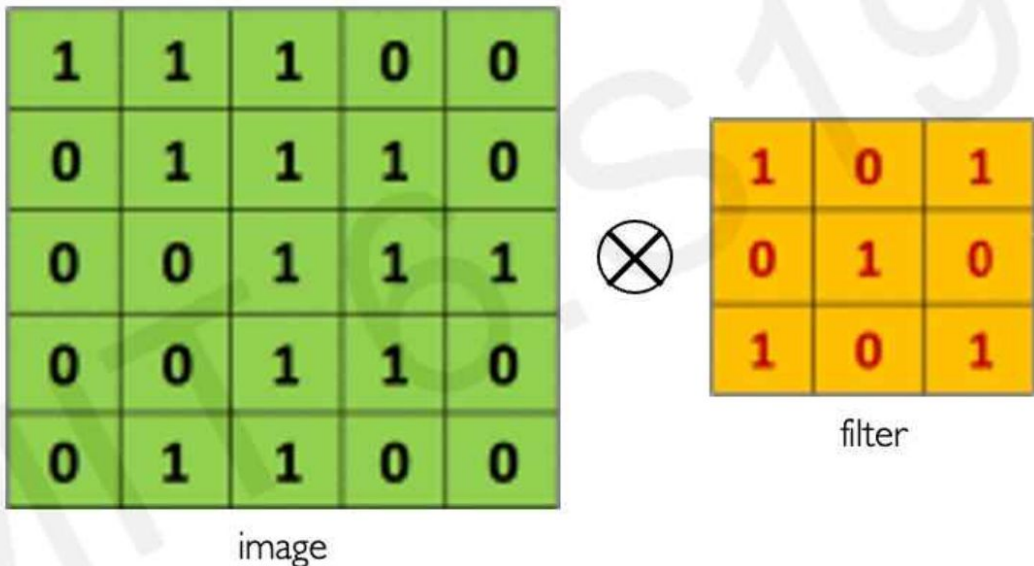
Edge Detect



“Strong” Edge
Detect

The Convolution Operation

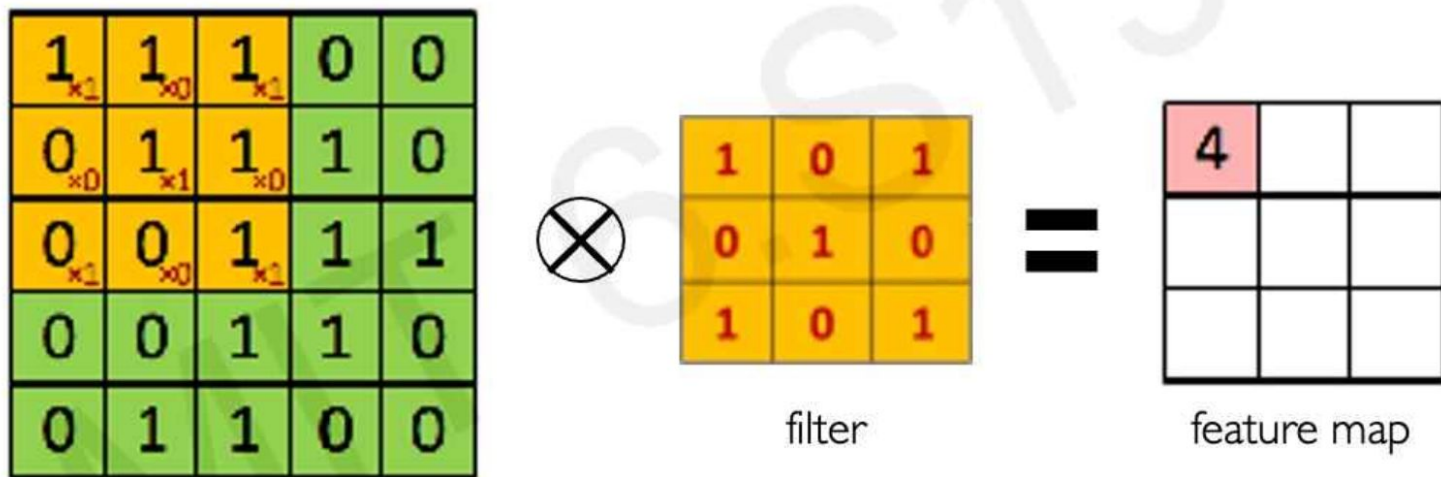
Suppose we want to compute the convolution of a 5x5 image and a 3x3 filter:



We slide the 3x3 filter over the input image, element-wise multiply, and add the outputs...

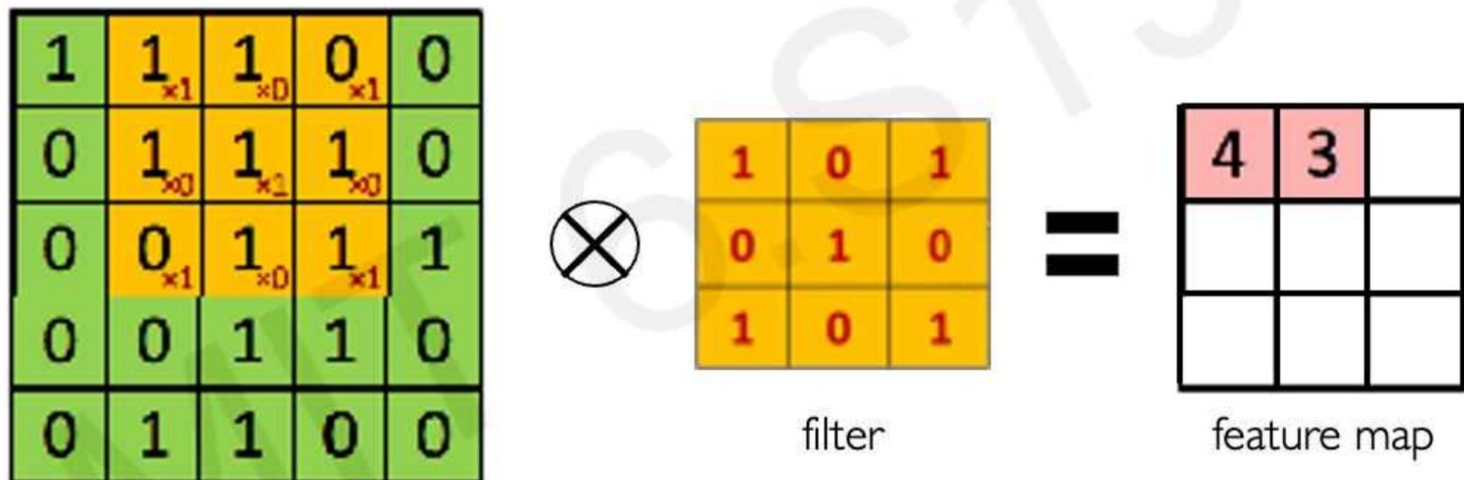
The Convolution Operation

We slide the 3x3 filter over the input image, element-wise multiply, and add the outputs:



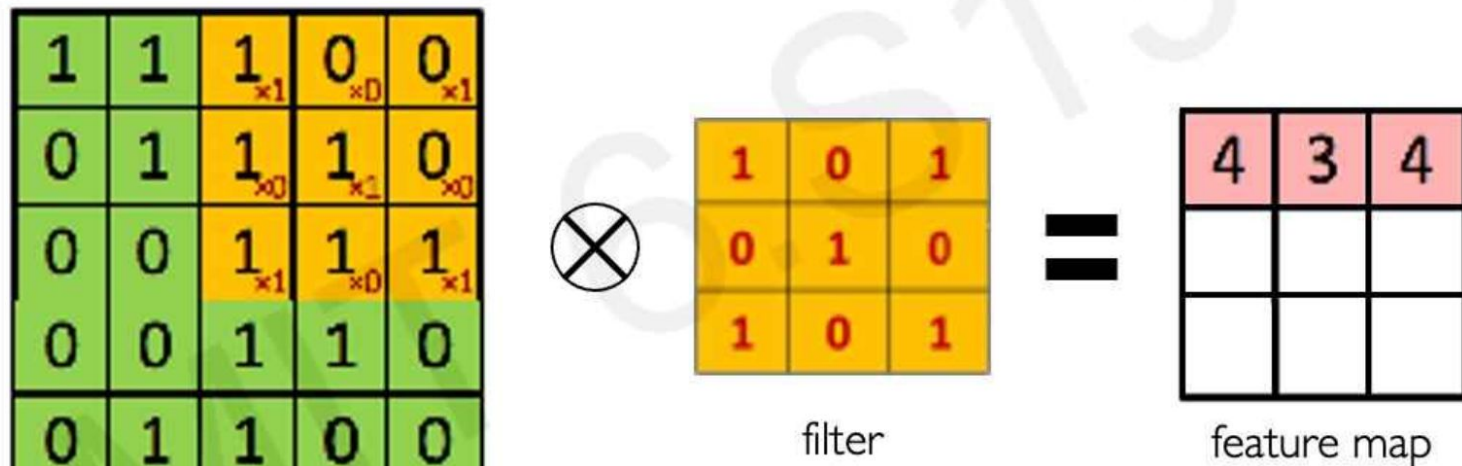
The Convolution Operation

We slide the 3x3 filter over the input image, element-wise multiply, and add the outputs:



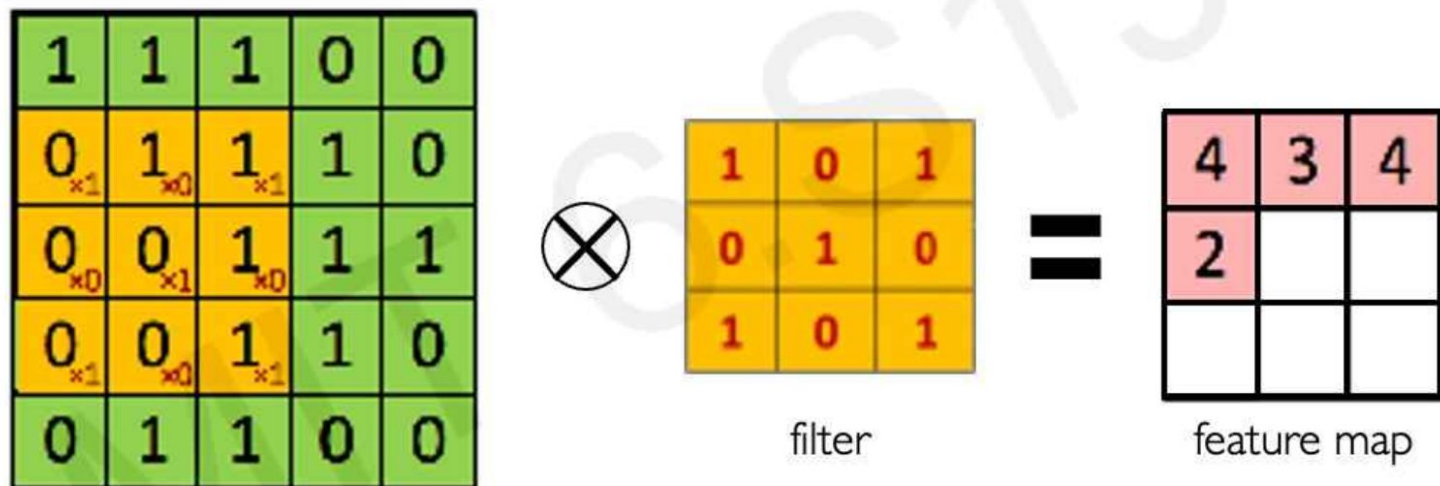
The Convolution Operation

We slide the 3x3 filter over the input image, element-wise multiply, and add the outputs:



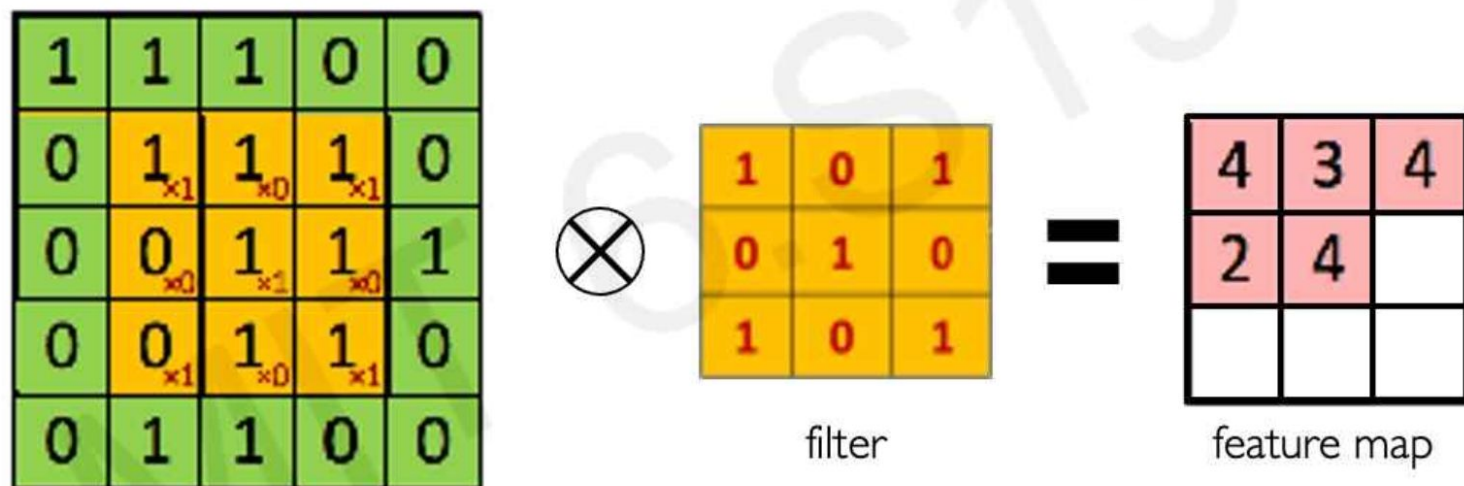
The Convolution Operation

We slide the 3x3 filter over the input image, element-wise multiply, and add the outputs:



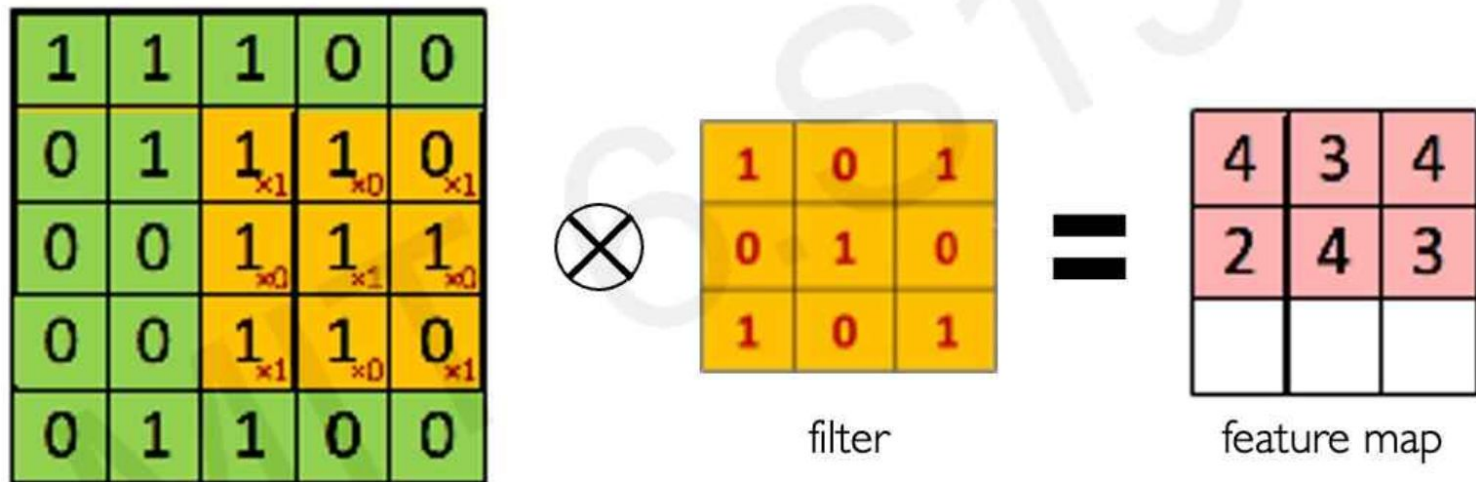
The Convolution Operation

We slide the 3x3 filter over the input image, element-wise multiply, and add the outputs:



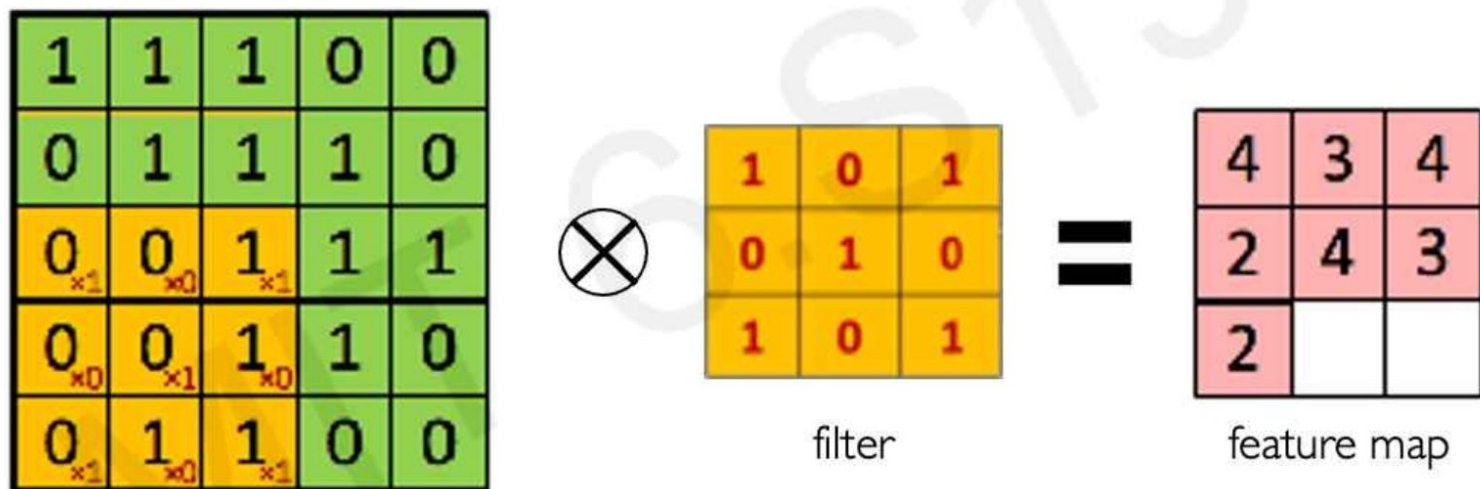
The Convolution Operation

We slide the 3x3 filter over the input image, element-wise multiply, and add the outputs:



The Convolution Operation

We slide the 3x3 filter over the input image, element-wise multiply, and add the outputs:



The Convolution Operation

We slide the 3x3 filter over the input image, element-wise multiply, and add the outputs:

1	1	1	0	0
0	1	1	1	0
0	0 _{x1}	1 _{x0}	1 _{x1}	1
0	0 _{x0}	1 _{x1}	1 _{x0}	0
0	1 _{x1}	1 _{x0}	0 _{x1}	0



1	0	1
0	1	0
1	0	1

filter

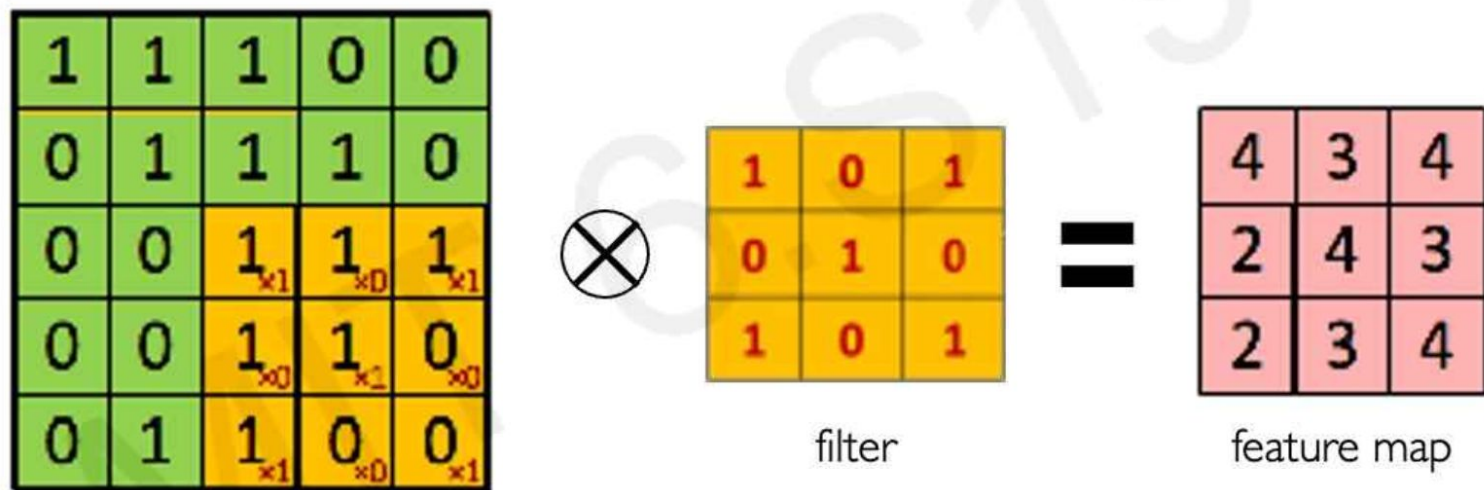


4	3	4
2	4	3
2	3	

feature map

The Convolution Operation

We slide the 3x3 filter over the input image, element-wise multiply, and add the outputs:



Filter dimensions

- For a grayscale image (represented as 2D matrix of pixels), we have 2D filters.
- Like 3x3, 5x5, etc.
- For 3-channel image (RGB), we have 3D filters (like 3D volume).
- 3D filters are basically stacked 2D filters.
- One 2D filter per **channel**.

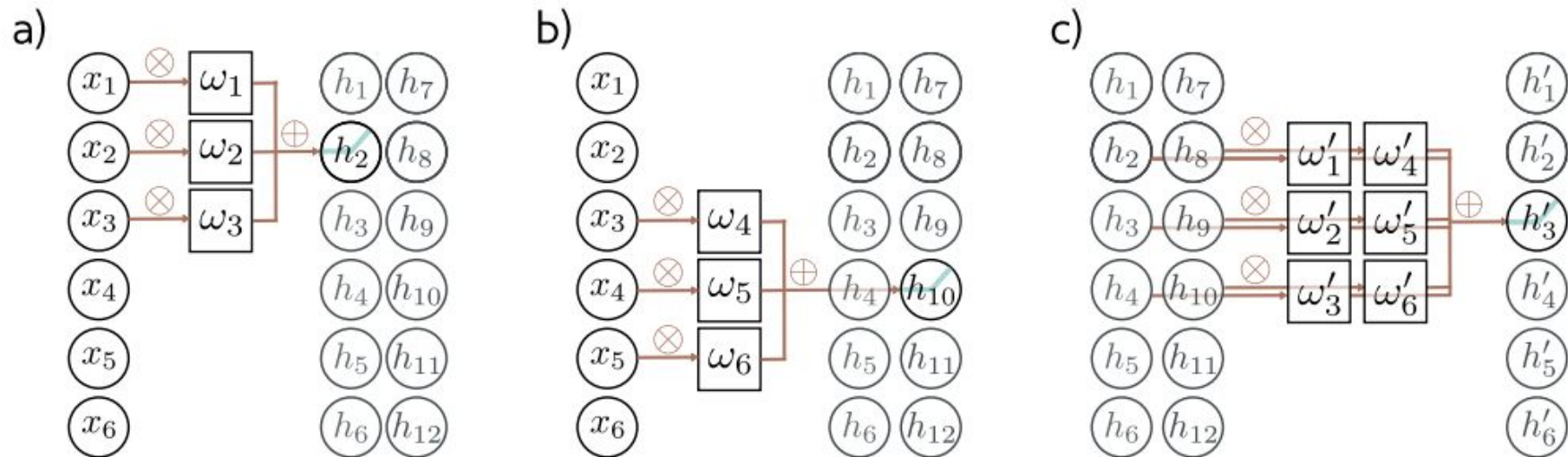


Figure 10.5 Channels. Typically, multiple convolutions are applied to the input $\mathbf{x}$ and stored in channels. a) A convolution is applied to create hidden units h_1 to h_6 , which form the first channel. b) A second convolution operation is applied to create hidden units h_7 to h_{12} , which form the second channel. The channels are stored in a 2D array $\mathbf{H}_1$ that contains all the hidden units in the first hidden layer. c) If we add a further convolutional layer, there are now two channels at each input position. Here, the 1D convolution defines a weighted sum over both input channels at the three closest positions to create each new output channel.

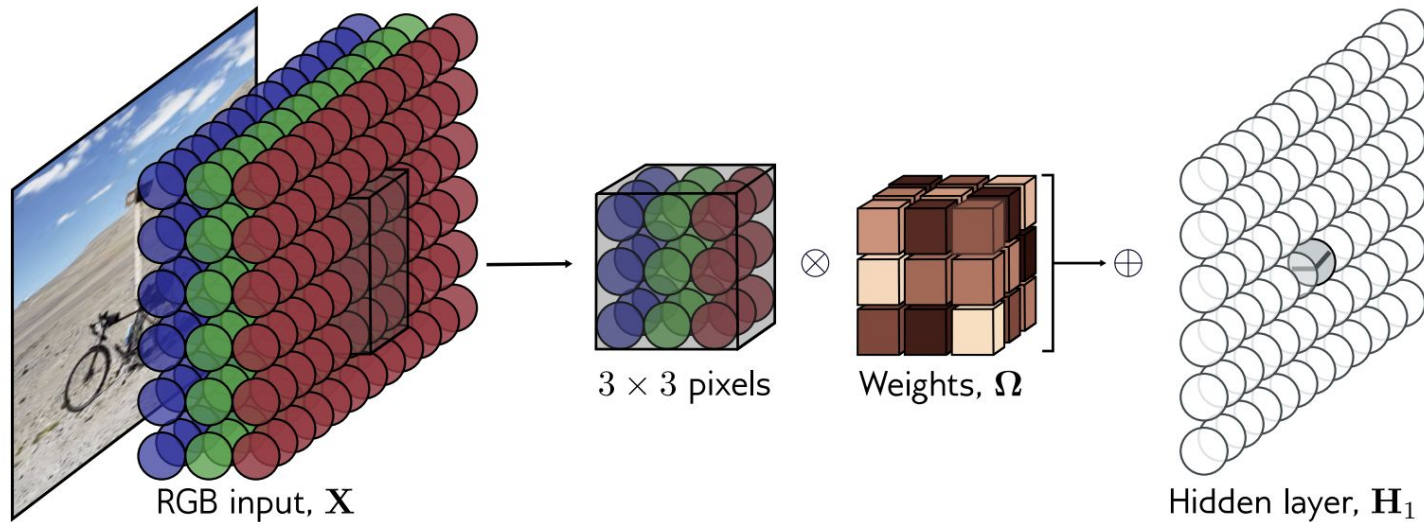
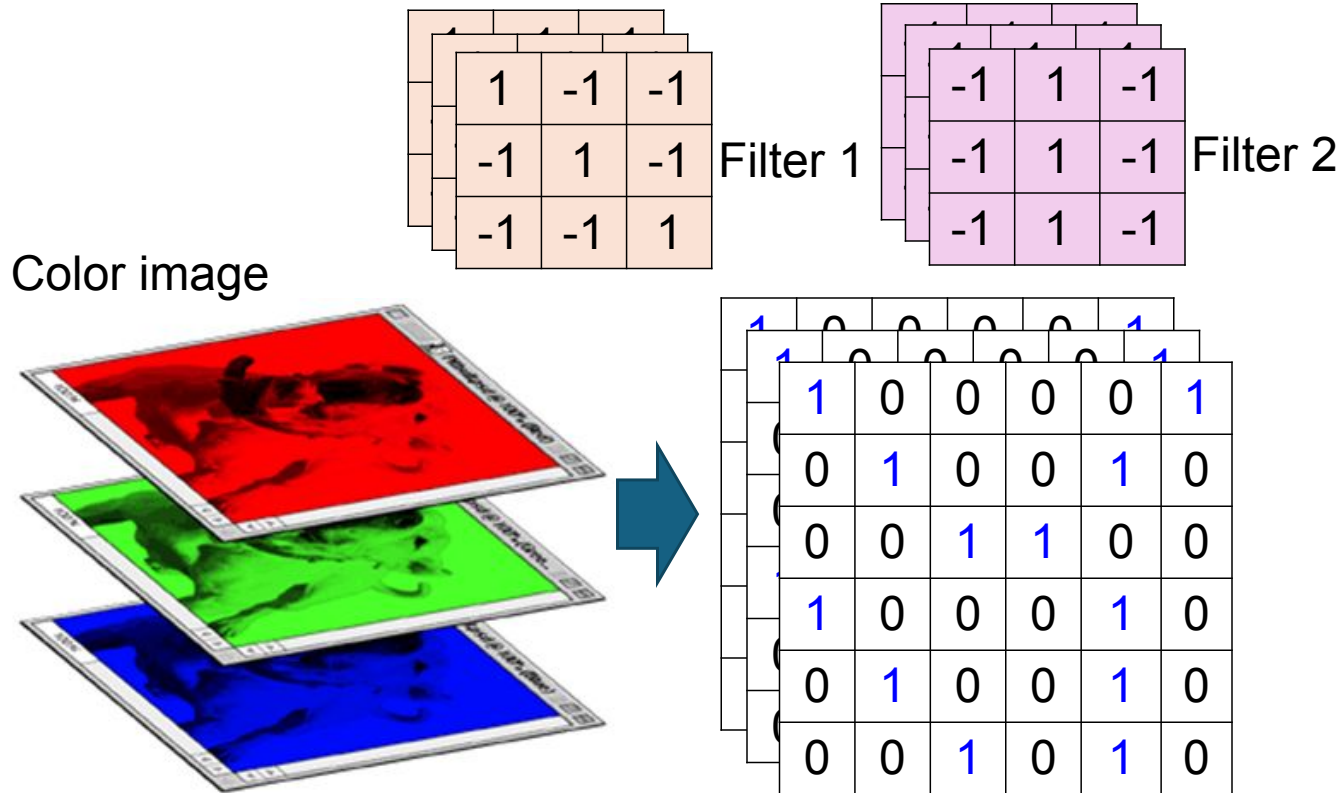


Figure 10.10 2D convolution applied to an image. The image is treated as a 2D input with three channels corresponding to the red, green, and blue components. With a 3×3 kernel, each pre-activation in the first hidden layer is computed by pointwise multiplying the $3 \times 3 \times 3$ kernel weights with the 3×3 RGB image patch centered at the same position, summing, and adding the bias. To calculate all the pre-activations in the hidden layer, we “slide” the kernel over the image in both horizontal and vertical directions. The output is a 2D layer of hidden units. To create multiple output channels, we would repeat this process with multiple kernels, resulting in a 3D tensor of hidden units at hidden layer H_1 .

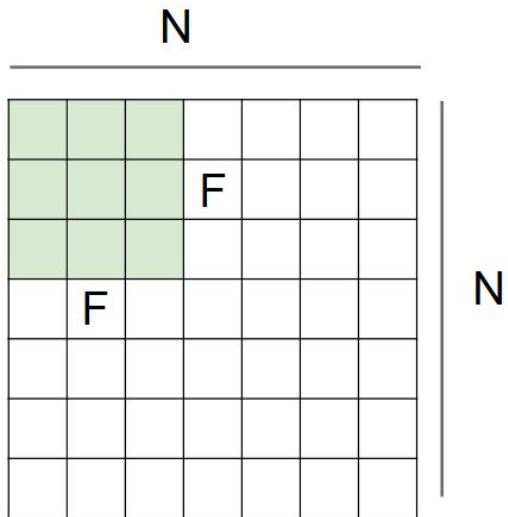
Color image: RGB 3 channels



Dimensions, Padding and Pooling

Spatial dimensions

Output size with different
strides.



Output size:

$$(N - F) / \text{stride} + 1$$

e.g. $N = 7$, $F = 3$:

$$\text{stride } 1 \Rightarrow (7 - 3) / 1 + 1 = 5$$

$$\text{stride } 2 \Rightarrow (7 - 3) / 2 + 1 = 3$$

$$\text{stride } 3 \Rightarrow (7 - 3) / 3 + 1 = 2.33$$

In practice: Common to zero pad

0	0	0	0	0	0			
0								
0								
0								
0								

e.g. input 7x7

3x3 filter, applied with **stride 1**

pad with 1 pixel border => what is the output?

(recall:)

$$(N - F) / \text{stride} + 1$$

In practice: Common to zero pad

0	0	0	0	0	0			
0								
0								
0								
0								

e.g. input 7x7

3x3 filter, applied with **stride 1**

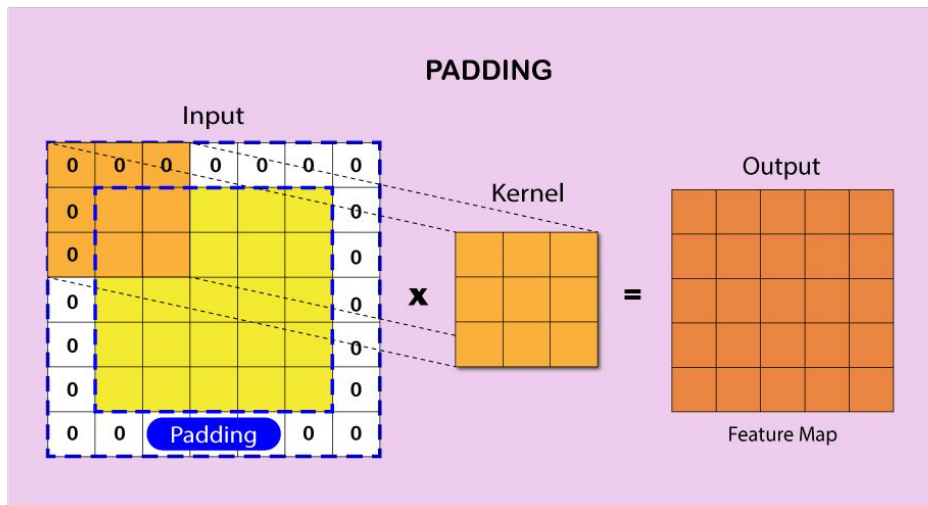
pad with 1 pixel border => what is the output?

Output size: 7 x
7

(recall:)

$(N - F) / \text{stride} + 1$

In practice: Common to zero pad



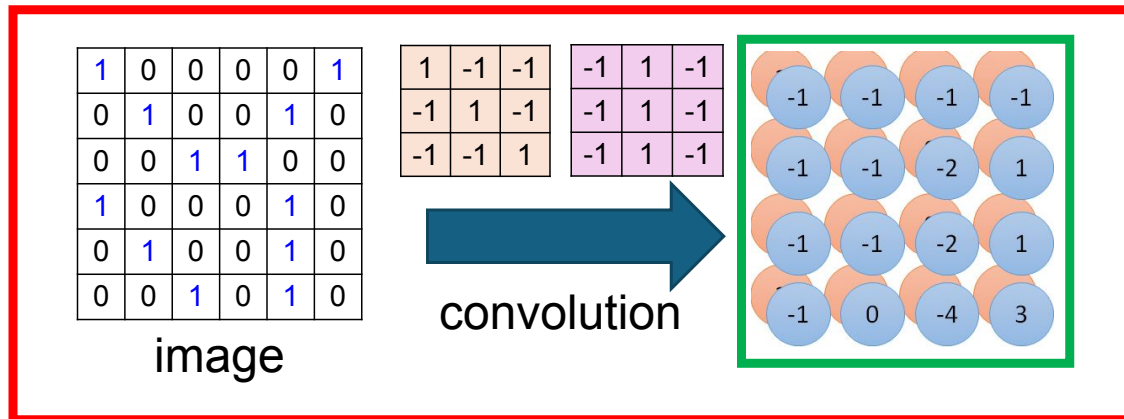
It is common to see CONV layers with stride 1, Filter $F \times F$ and zero-padding with $(F-1)/2$.

Zero-padding helps preserve the size spatially.

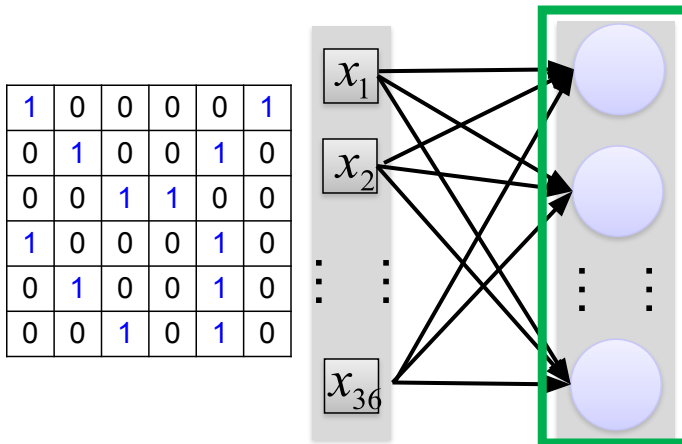
Zero-padding has two purposes:

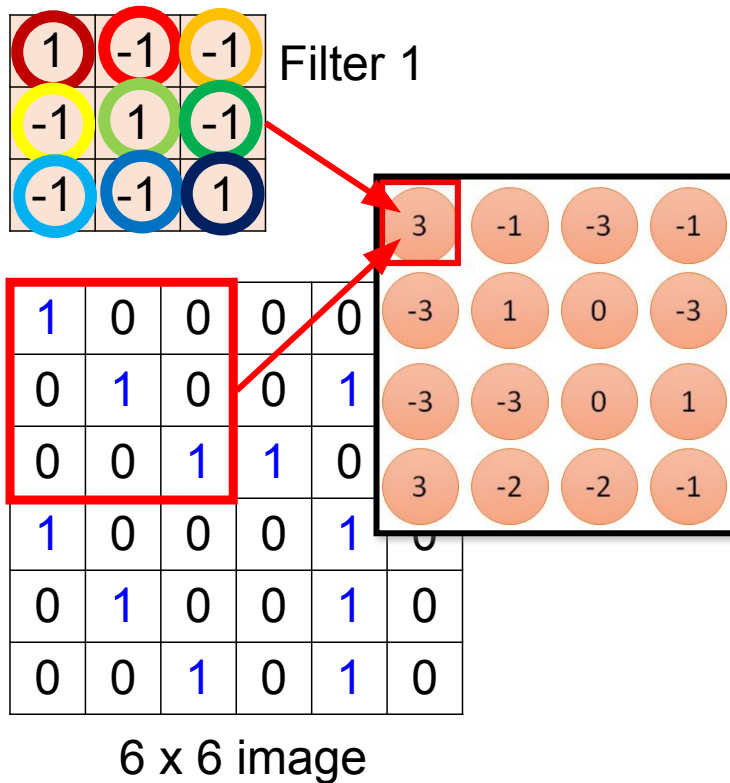
- Artificially increase input size to prevent over-shrinkage after repeated convolution.
- During the convolution process, the pixels at the corner are barely looked at.
- Central pixels are repeatedly calculated, and network becomes almost blind to corners.

Convolution v.s. Fully Connected

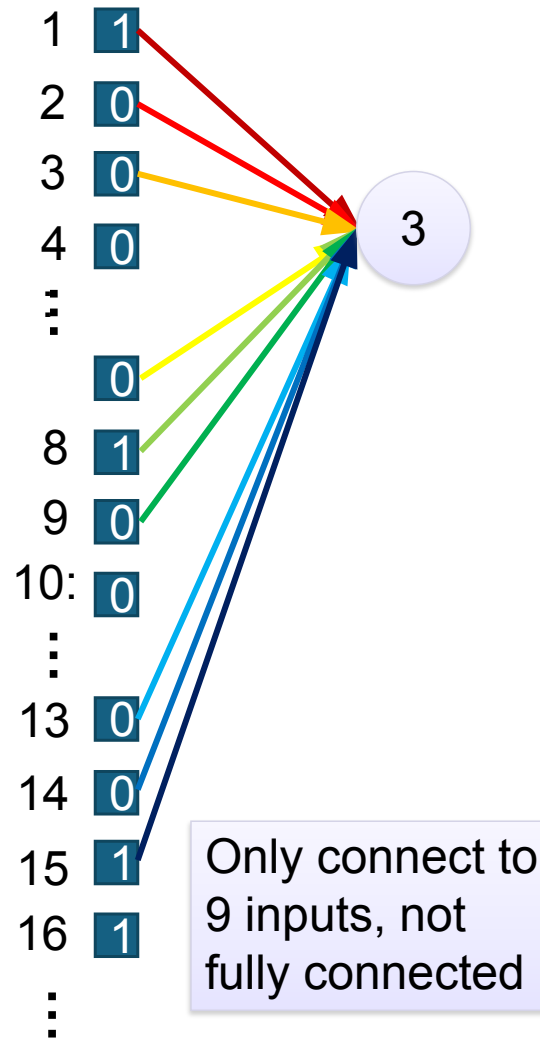


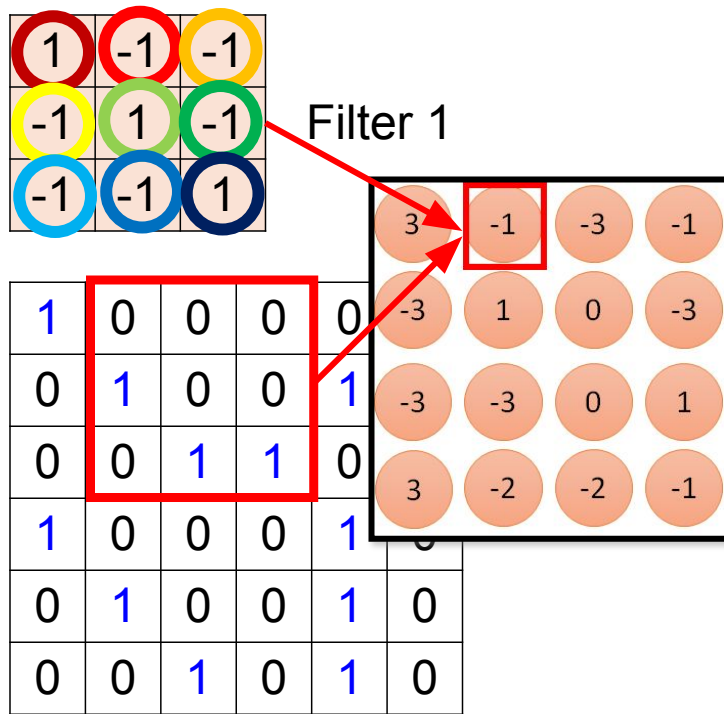
Fully-connected





fewer parameters!

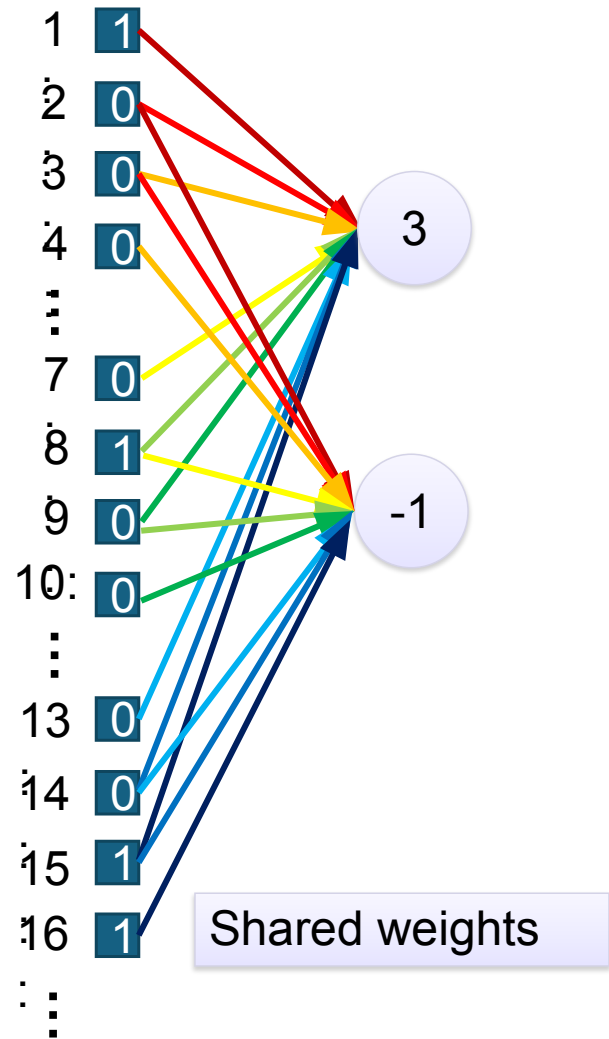




6 x 6 image

Fewer parameters

Even fewer parameters



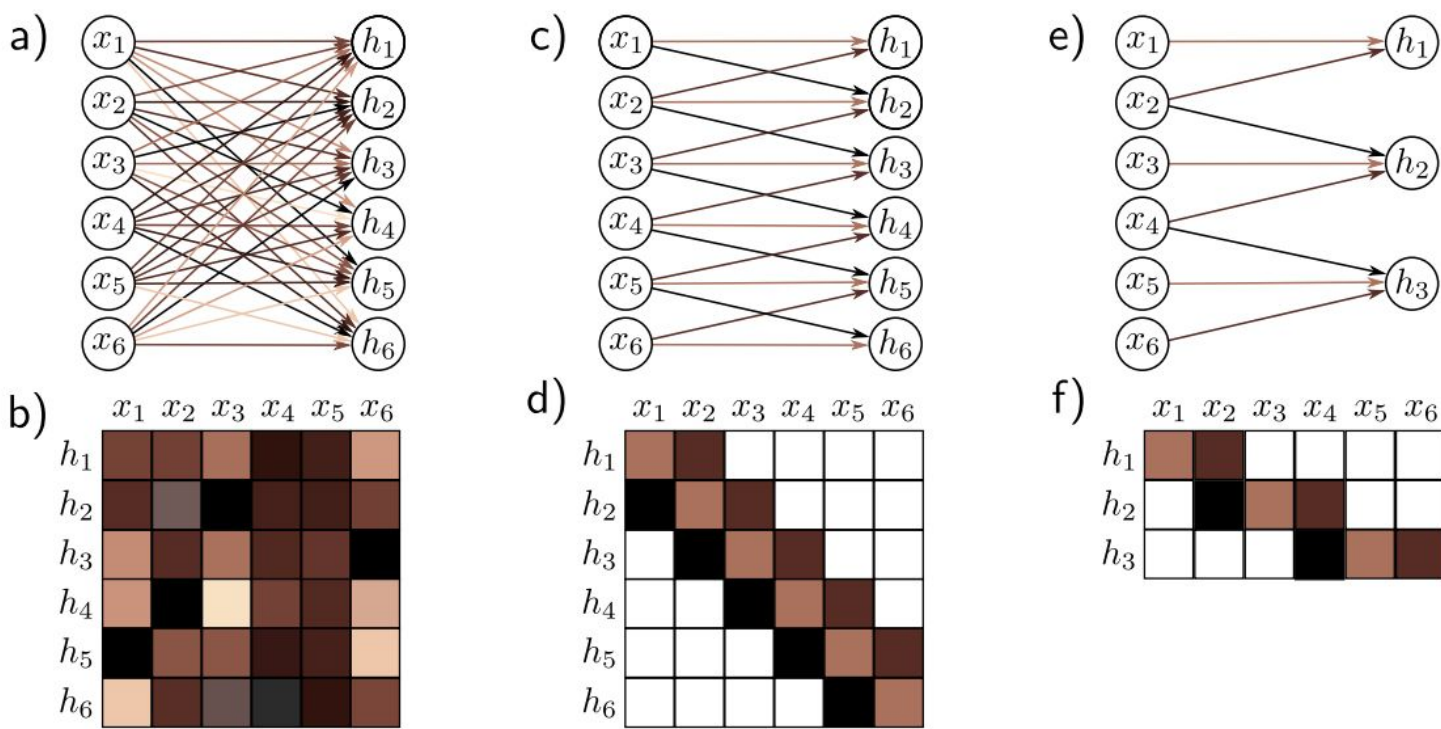
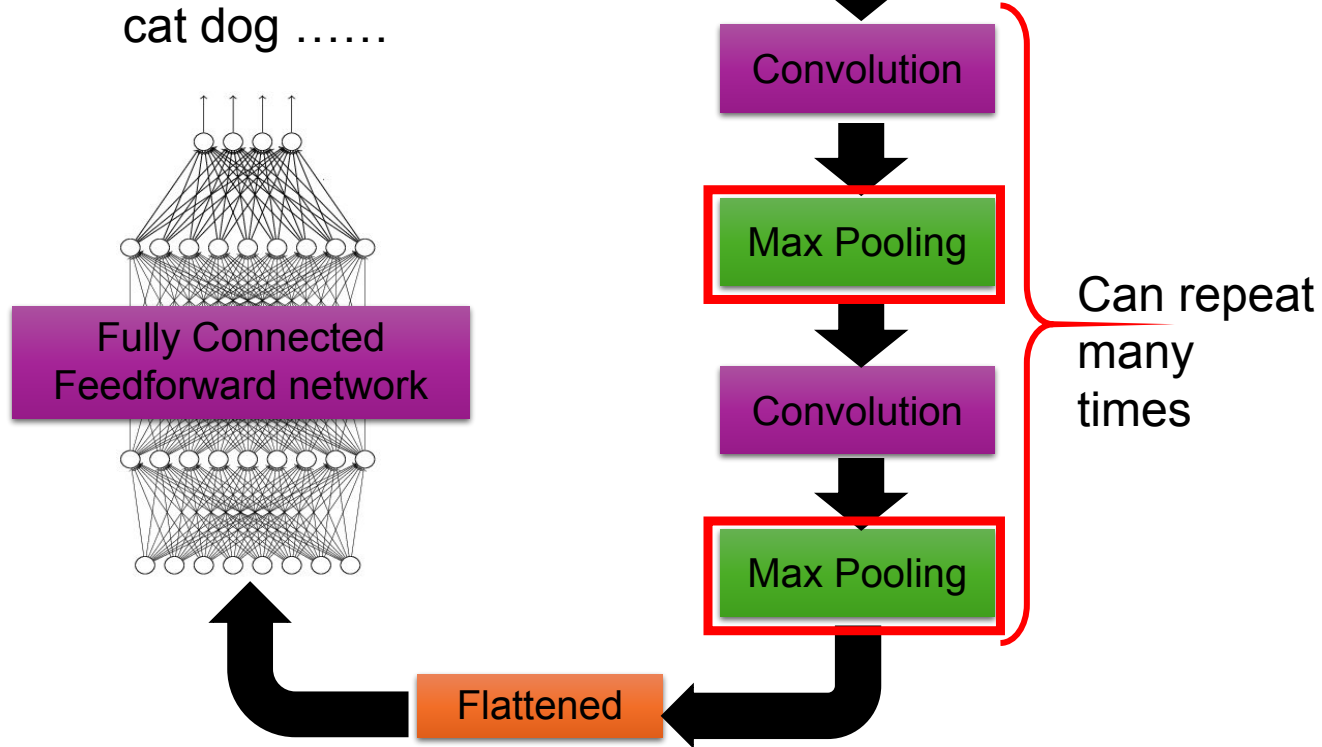


Figure 10.4 Fully connected vs. convolutional layers.

- a) A fully connected layer has a weight connecting each input x to each hidden unit h (colored arrows) and a bias for each hidden unit (not shown).
- b) Hence, the associated weight matrix Ω contains 36 weights relating the six inputs to the six hidden units.
- c) A convolutional layer with kernel size three computes each hidden unit as the same weighted sum of the three neighboring inputs (arrows) plus a bias (not shown).
- d) The weight matrix is a special case of the fully connected matrix where many weights are zero and others are repeated (same colors indicate same value, white indicates zero weight).
- e) A convolutional layer with kernel size three and stride two computes a weighted sum at every other position.
- f) This is also a special case of a fully connected network with a different sparse weight structure.

The whole CNN



Max Pooling

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

-1	1	-1
-1	1	-1
-1	1	-1

Filter 2

3	-1	-3	-1
-3	1	0	-3
-3	-3	0	1
3	-2	-2	-1

-1	-1	-1	-1
-1	-1	-2	1
-1	-1	-2	1
-1	0	-4	3

Why Pooling

- Subsampling pixels will not change the object

bird



Subsampling

bird



We can subsample the pixels to make image smaller



fewer parameters to characterize the image

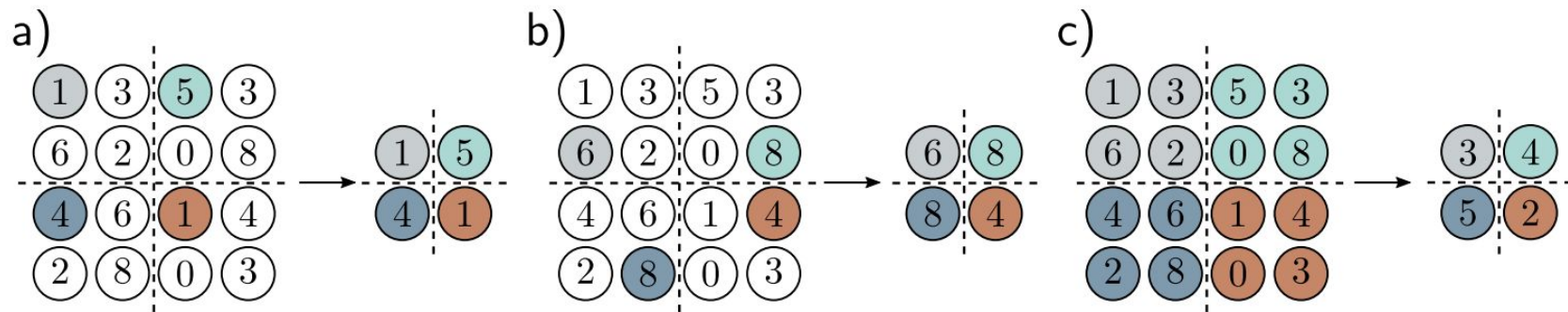


Figure 10.11 Methods for scaling down representation size (downsampling). a) Sub-sampling. The original 4×4 representation (left) is reduced to size 2×2 (right) by retaining every other input. Colors on the left indicate which inputs contribute to the outputs on the right. This is effectively what happens with a kernel of stride two, except that the intermediate values are never computed. b) Max pooling. Each output comprises the maximum value of the corresponding 2×2 block. c) Mean pooling. Each output is the mean of the values in the 2×2 block.

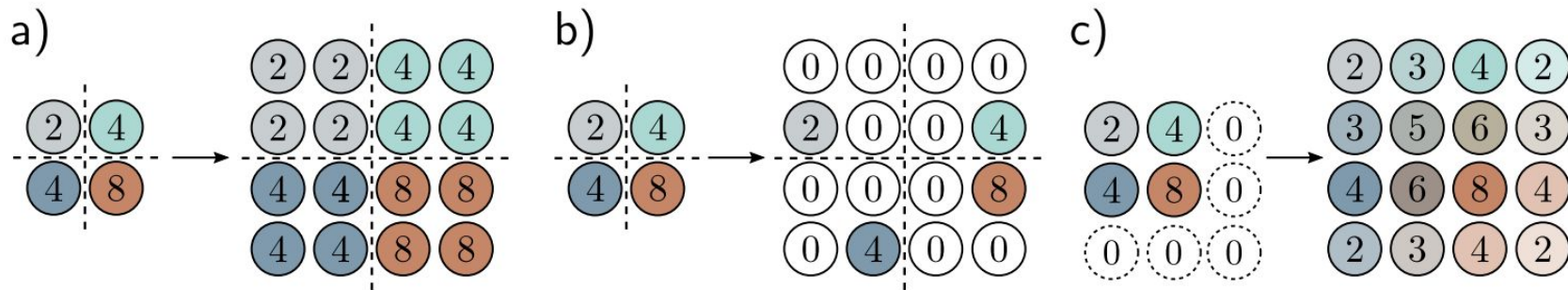
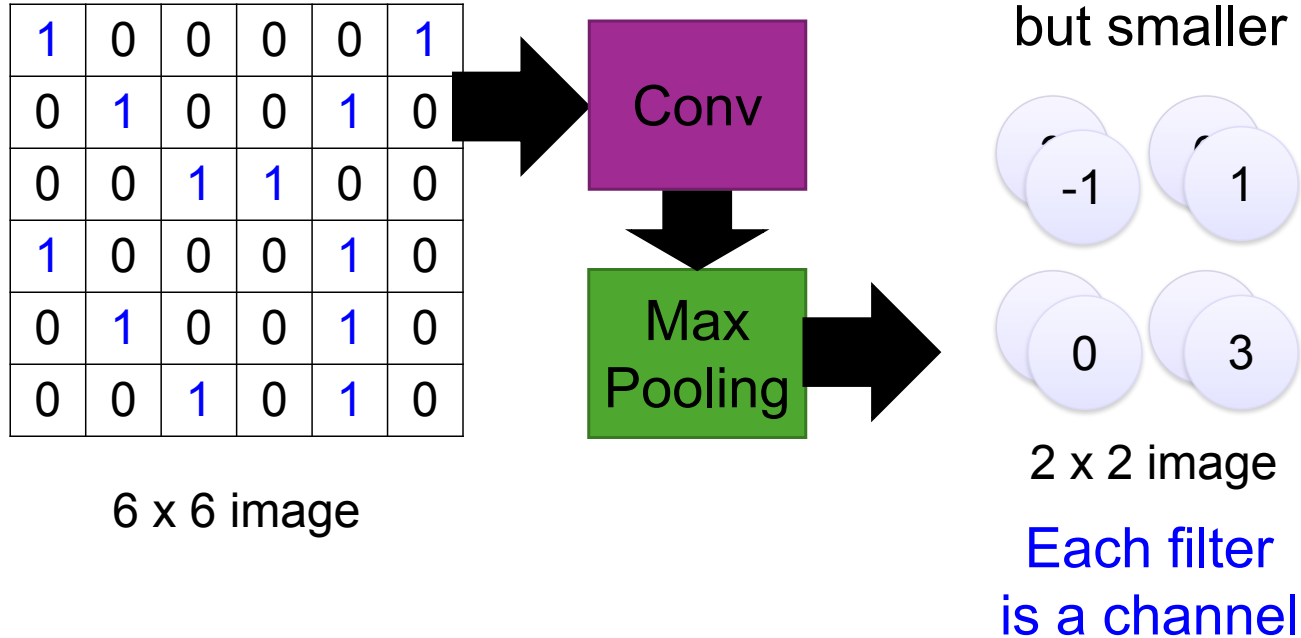
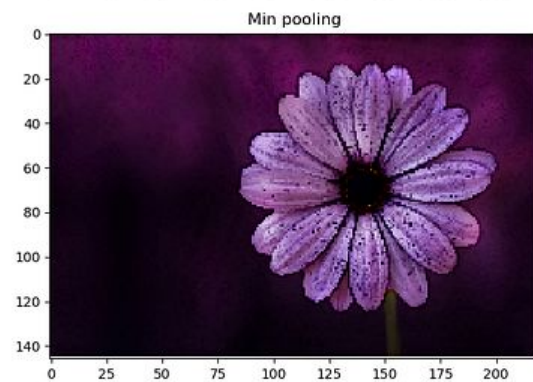
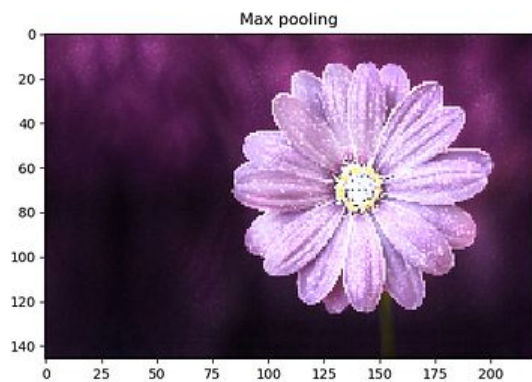
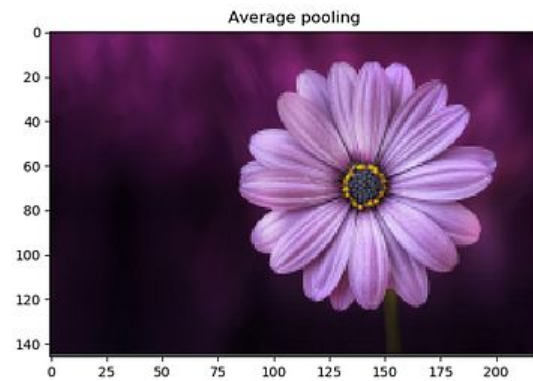


Figure 10.12 Methods for scaling up representation size (upsampling). a) The simplest way to double the size of a 2D layer is to duplicate each input four times. b) In networks where we have previously used a max pooling operation (figure 10.11b), we can redistribute the values to the same positions they originally came from (i.e., where the maxima were). This is known as max unpooling. c) A third option is bilinear interpolation between the input values.

Max Pooling





A CNN compresses a fully connected network by:

- Reducing number of connections
- Shared weights
- Max pooling further reduces the complexity

Receptive field

- Receptive field is defined as the size of the region in the input that produces the feature.
- In ANN, every hidden unit has access to whole input.
- In CNNs, the RF of units in successive layers increases.

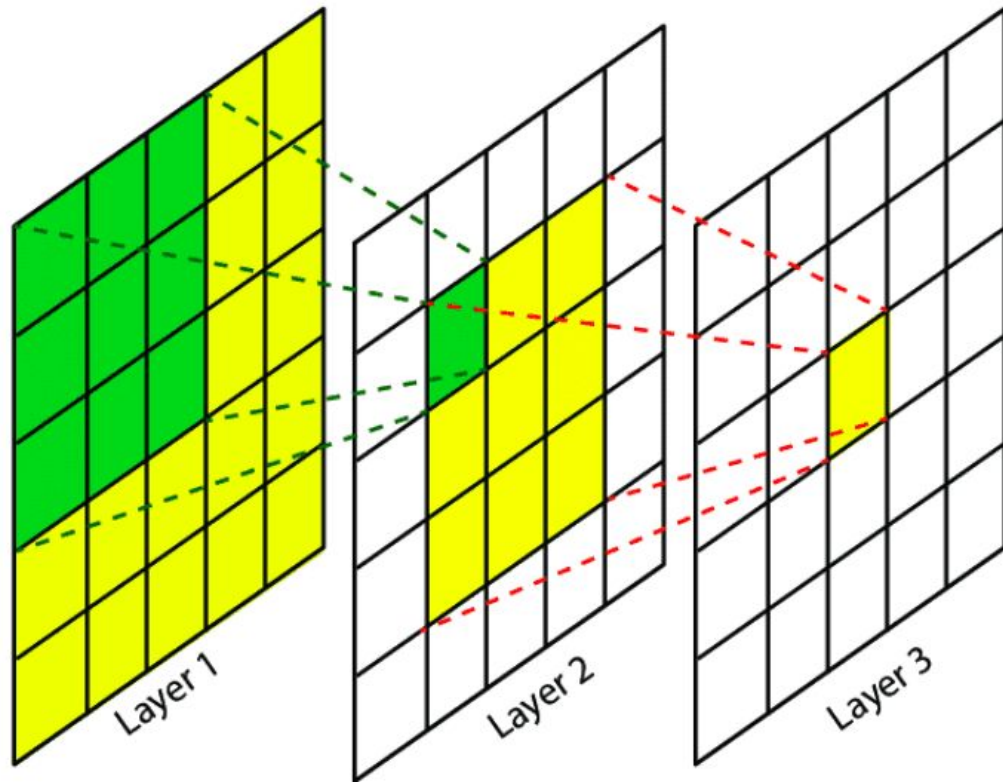
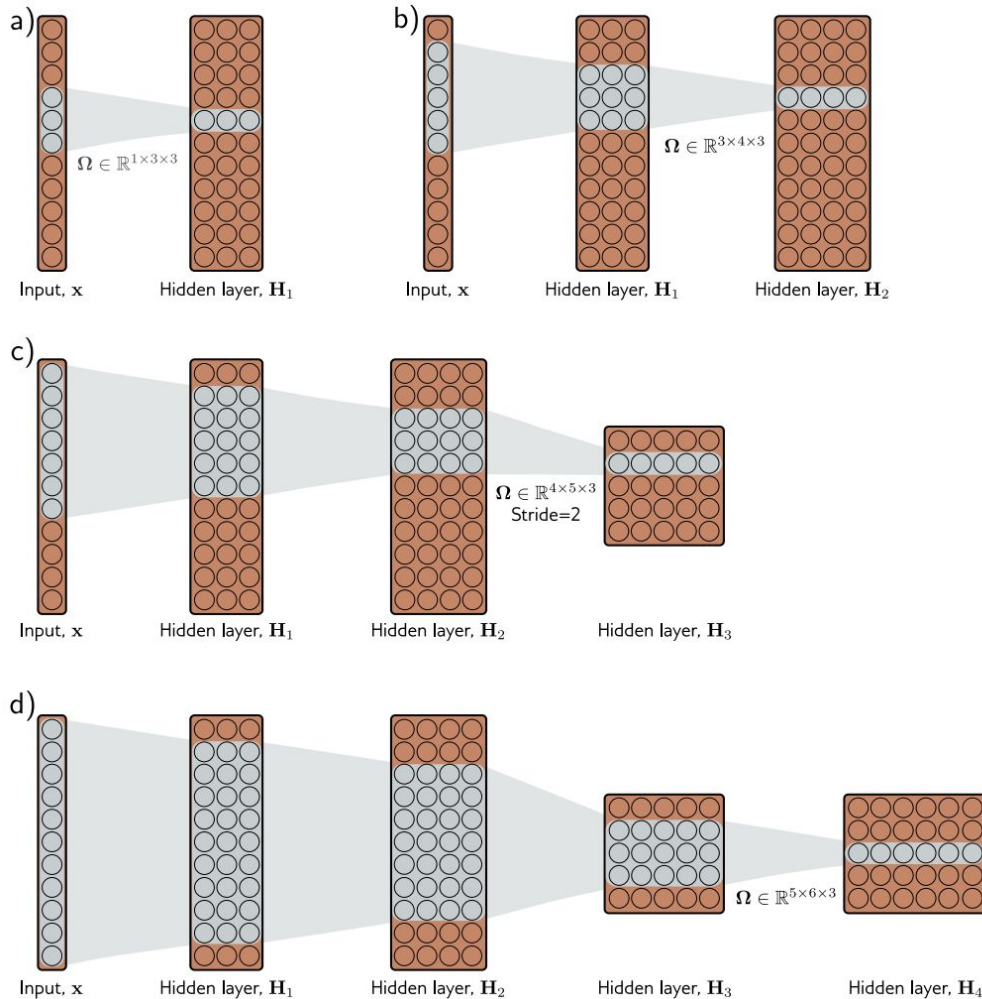


Figure 10.6 Receptive fields for network with kernel width of three.



a) An input with eleven dimensions feeds into a hidden layer with three channels and convolution kernel of size three. The pre-activations of the three highlighted hidden units in the first hidden layer H_1 are different weighted sums of the nearest three inputs, so the receptive field in H_1 has size three.

b) The pre-activations of the four highlighted hidden units in layer H_2 each take a weighted sum of the three channels in layer H_1 at each of the three nearest positions. Each hidden unit in layer H_1 weights the nearest three input positions. Hence, hidden units in H_2 have a receptive field size of five.

c) The hidden units in the third layer (kernel size three, stride two) increases the receptive field size to seven.

d) By the time we add a fourth layer, the receptive field of the hidden units at position three have a receptive field that covers the entire input.